

Faculty of Engineering
Alexandria University

6-DOF Robotic Arm

Trajectory Planning / Simulation / Hardware Control

Submitted by:

Khaled Ahmed Elwan - ID: 8782

Taha Ahmed Elaghoury - ID: 8745

Marwan Mahmoud Ali Bakhit - ID: 9023

Supervised by:

Dr. Sohair Rezeka



1. Introduction

This report presents the complete design, simulation, trajectory planning, and hardware implementation of a 6 Degrees-of-Freedom (6-DOF) robotic arm. The project integrates a multi-tool engineering workflow that covers the full development cycle, from mechanical CAD modeling and physics-based simulation to motion planning, custom electronics, and real-time servo actuation.

The project was developed by Khaled Ahmed Elwan (ID: 8782), Taha Ahmed Elaghoury (ID: 8745), and Marwan Mahmoud Ali Bakhit (ID: 9023), under the supervision of Dr. Sohair Rezeka at the Faculty of Engineering, Alexandria University.

The completed project stages are summarized as follows:

- Mechanical design of the complete robotic arm using SolidWorks
- Physics-based simulation using MATLAB Simscape Multibody
- Polynomial trajectory planning with six configurable Cartesian waypoints
- Inverse Kinematics (IK) solution using the MATLAB Robotics System Toolbox
- 3D end-effector path animation in MATLAB
- Robot workspace visualization and analysis
- Real-time motion planning and simulation using ROS2 Jazzy and MoveIt2 in RViz2
- Custom PCB design for servo motor interfacing and power distribution
- Hardware servo control using an ESP32-WROOM microcontroller

2. System Overview

The robotic arm was developed as a serial manipulator with revolute joints. The prototype includes five primary arm joints and an actuated gripper, providing the controlled motion required for positioning, orientation adjustment, and object handling. The system combines mechanical design, simulation, motion planning, and embedded control into a single integrated platform.

Parameter	Description
Degrees of Freedom	6 DOF
Joint Configuration	Revolute joints
Joint Names	Shoulder, Elbow, Wrist, Rolling, Pitch, Gripper
Microcontroller	ESP32-WROOM DevKitC
Number of Servos	5 active servos + 1 gripper servo
Servo Signal Pins	D13, D12, D14, D27, D26, D25
Control Software	MATLAB R2024a
Simulation Engine	Simscape Multibody
Motion Planner	Polynomial trajectory generation + IK solver
ROS Version	ROS2 Jazzy
Motion Planning Framework	MoveIt2
PCB Design Tool	KiCad
CAD Tool	SolidWorks

3. Mechanical Design (SolidWorks)

The complete robotic arm was designed from scratch using SolidWorks. Each component, including the links, joint housings, base, wrist elements, and gripper end-effector, was modeled individually and assembled with the required mechanical constraints to represent the intended joint motion.

The mechanical design workflow included the following steps:

- Individual part modeling for each link, joint housing, and gripper component
- Full assembly of the robotic arm with revolute joint constraints
- Definition of coordinate systems at the relevant joint and end-effector locations
- Gripper end-effector design for object grasping
- Export of the mechanical model for MATLAB Simscape Multibody simulation
- Export of the robot model for ROS2 and MoveIt2 integration

Figures 1 and 2 show the final SolidWorks assembly and the rendered CAD model of the complete robotic arm.

3.1 SolidWorks Modeling

In SolidWorks, the robotic arm assembly was constructed and adjusted to ensure accurate part positioning, joint alignment, and link connectivity. Coordinate systems were assigned to the individual components at the joint locations to define the relative motion between consecutive links and to support the subsequent export to Simscape Multibody.

Special attention was given to the end-effector coordinate system. This frame was placed within the end-effector sub-assembly rather than directly on individual gripper parts because assigning the frame to certain parts produced errors during the Simscape export process. Defining the frame at the sub-assembly level provided a more stable reference for the end-effector and improved the reliability of the exported model.

The mechanical assembly includes five main revolute joints responsible for the arm motion, in addition to the joints associated with the gripper mechanism. Each joint was designed to rotate about a defined axis. The base joint provides horizontal rotation of the arm, the shoulder and elbow joints provide vertical lifting and extension, the wrist joints provide end-effector orientation control, and the gripper enables object grasping and holding.

3.2 Fabrication

The robotic arm components were fabricated using fused-filament fabrication (FFF) on a Bambu P1S 3D printer. PLA was selected as the structural material, and a 50% infill density was applied to the printed parts.

PLA was selected because it offers high stiffness, low printing shrinkage, limited warping, and reliable dimensional stability compared with more flexible or difficult-to-print polymers. These properties make it suitable for functional robotic prototypes where accurate assembly, joint alignment, and repeatable motion are required. The 50% infill density was selected as an engineering compromise between rigidity, part mass, printing time, and material consumption. This infill level provides internal support and improves resistance to bending and torsional loads in the links and joint brackets, while avoiding unnecessary mass that would increase joint inertia and servo torque demand.

During the first fabrication trial, a print failure occurred, as indicated by the accumulation of loose extruded filament on the build plate. This failure mode is commonly associated with loss of first-layer adhesion, insufficient support, inaccurate bed leveling, or unstable bonding between the deposited

filament and the build surface. Once the part detached or lost support, the nozzle continued extruding material without forming the intended geometry, resulting in a typical spaghetti-type FFF failure. The issue was addressed by checking bed leveling and adhesion, reviewing the slicing parameters, and verifying the required supports before reprinting the part.



Initial FFF printing failure observed during the first fabrication trial.

3.3 Coordinate Axes and SMILINK Structure

During the SolidWorks-to-Simscape export process, the model generated a hierarchical Simscape Multibody Link structure, referred to as the SMILINK structure. This structure defines the relationships between the robot components, joints, and coordinate frames. It represents the connection sequence from the fixed world frame through the complete kinematic chain until the end-effector.

The exported model included several SMILINK frames, starting from the base reference and continuing through the arm links, wrist components, and end-effector. These frames define the relative position and orientation of each body with respect to the adjacent body. A clear coordinate hierarchy is essential for preserving the spatial relationship between the links and for enabling accurate forward and inverse kinematic calculations in MATLAB.

The gripper mechanism required additional connection points because it contains multiple moving components. These additional frames were necessary to preserve the coordinated motion of the gripper fingers during opening and closing while maintaining correct alignment with the end-effector reference frame.

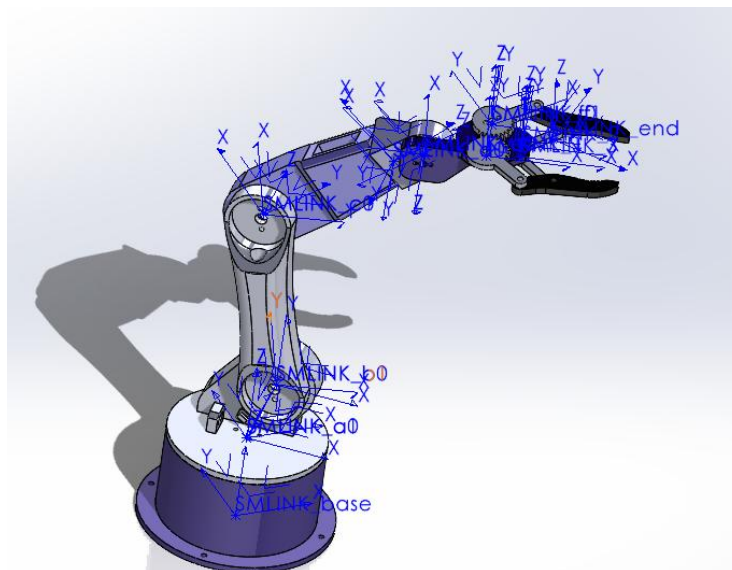


Figure 5: Simscape Multibody imported assembly showing link coordinate frames and end-effector reference frame.

3.4 Export to Simscape Multibody

The Simscape Multibody Link add-in was used to export the completed SolidWorks assembly into MATLAB Simscape Multibody. The export process converted the mechanical assembly, including its parts, joints, coordinate systems, and geometry files, into a Simscape-compatible model. The generated output included an XML file describing the multibody structure, STL geometry files for visualization, and a MATLAB script used to import and construct the model inside Simscape Multibody.

A major export issue was encountered when the complete assembly was exported with the servo motor models included. The presence of the servo bodies complicated the joint definitions and caused the revolute joints to become incorrectly interpreted or difficult for the software to identify. As a result, the generated model did not accurately represent the intended joint structure.

To resolve this issue, the servo motor models were removed from the SolidWorks assembly before export, and the joints were connected directly between the mechanical links. This simplified the multibody structure, reduced export errors, and produced a cleaner Simscape model suitable for simulation, trajectory planning, and control implementation.



Figure 1: Final SolidWorks assembly of the 6-DOF robotic arm and gripper.

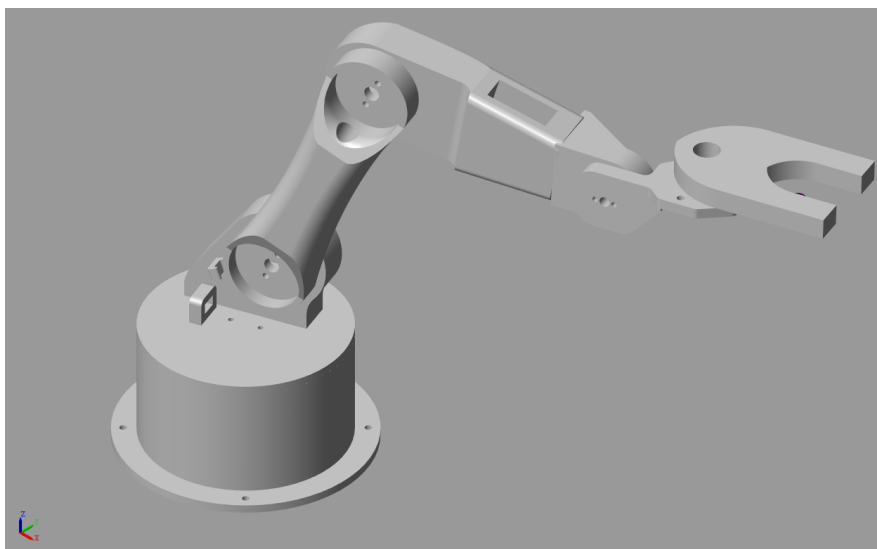


Figure 2: Rendered CAD model of the complete robotic arm assembly.

4. MATLAB Simscape Multibody Simulation

4.1 Model Architecture

The SolidWorks assembly was imported into MATLAB Simscape Multibody while preserving the main joint constraints, link geometries, and coordinate-frame relationships. The resulting model provides a physics-based representation of the robotic arm that can be connected directly to the trajectory-planning and control blocks in Simulink.

The Simulink model includes the following main blocks:

- Polynomial Trajectory block, which generates smooth joint-space or Cartesian trajectory profiles
- Coordinate Transformation Conversion block, which converts the trajectory output into SE(3) transformation matrices
- Inverse Kinematics (IK) Solver block, which computes the required joint angles from the desired Cartesian poses using the dof6 rigid-body-tree model
- myRobot Simscape block, which performs the physics-based simulation of the complete robotic arm
- Forward Kinematics (FK) verification block, which checks the end-effector position produced by the computed joint angles
- To Workspace blocks, which export the joint-angle data for post-processing and hardware control

Figure 3 shows the MATLAB/Simulink pipeline used for trajectory generation, coordinate transformation, IK solving, Simscape simulation, and FK verification.

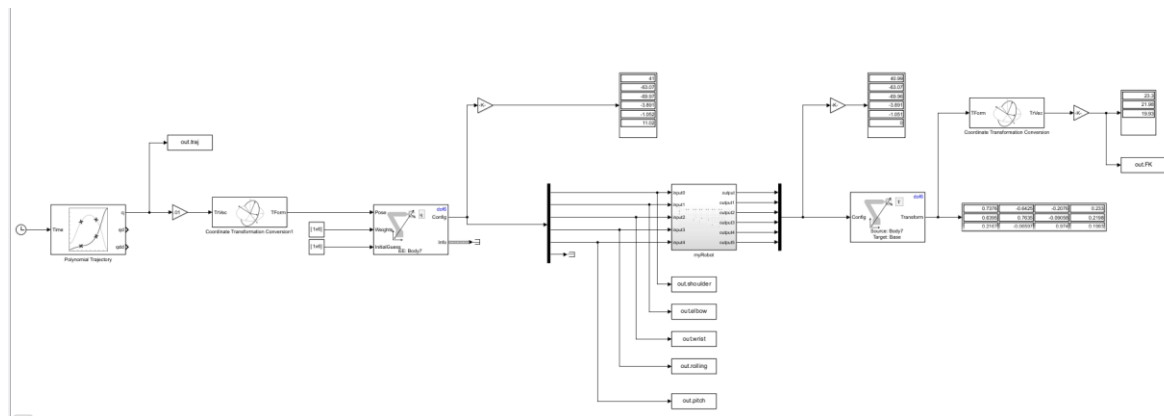


Figure 3: MATLAB/Simulink model architecture including polynomial trajectory generation, coordinate transformation, IK solving, Simscape simulation, and FK verification.

Figures 4 show the imported Simscape Multibody visualization

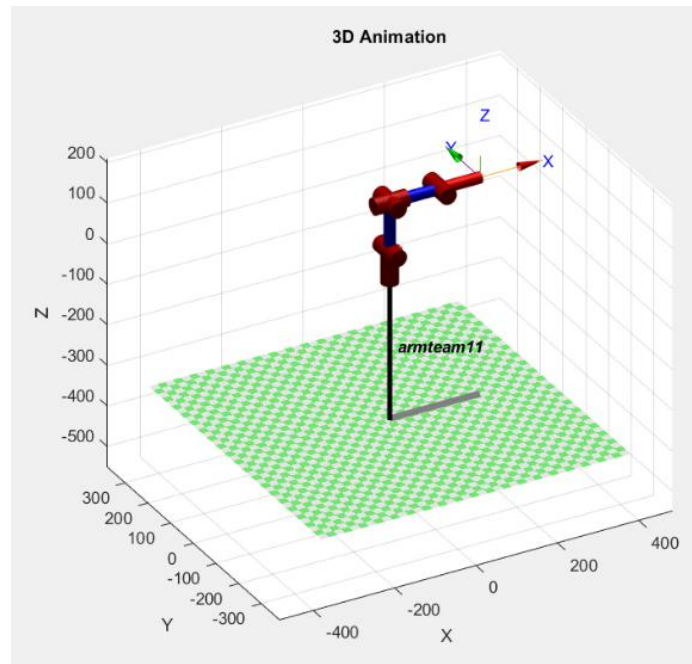


Figure 4: Simscape Multibody visualization of the robotic arm during 3D animation.

4.2 Trajectory Planning

Trajectory planning was implemented using fifth-order polynomial interpolation between user-defined Cartesian waypoints. The waypoints were defined as a 3 x N matrix representing the desired X, Y, and Z coordinates within the robot workspace. Zero boundary velocity conditions were applied at the start and end of each trajectory segment to produce smooth acceleration and deceleration behavior.

The IK solver operates on the dof6 rigid-body-tree model and computes the joint-angle vector [shoulder, elbow, wrist, rolling, pitch] required to place the end-effector at each point along the trajectory. Joint limits were considered during the IK computation to maintain physically feasible arm configurations throughout the motion.

4.3 Forward Kinematics Verification

The computed joint angles were passed through a Forward Kinematics block to verify the resulting end-effector pose. The FK output position was compared with the planned Cartesian trajectory to confirm that the IK solver produced valid joint configurations and that the simulated arm followed the intended path accurately.

5. Robot Workspace Analysis

The reachable workspace of the robotic arm was evaluated by uniformly sampling the permitted joint-angle ranges and calculating the corresponding end-effector positions in 3D Cartesian space. This method provides a practical visualization of the volume that the robot can reach while respecting the specified joint limits.

Joint	Minimum Angle	Maximum Angle
Shoulder	-90 degrees	+90 degrees
Elbow	-90 degrees	+90 degrees
Wrist	-90 degrees	+90 degrees
Rolling	-90 degrees	+90 degrees

Pitch	-90 degrees	+90 degrees
Gripper	-90 degrees	+90 degrees

The workspace analysis showed a near-spherical reachable volume with an approximate radius of 300 mm in the XY plane and a vertical Z-axis range extending from approximately -200 mm to +400 mm. A void region appears near the base due to mechanical self-collision constraints and the limited angular range of the shoulder joint.

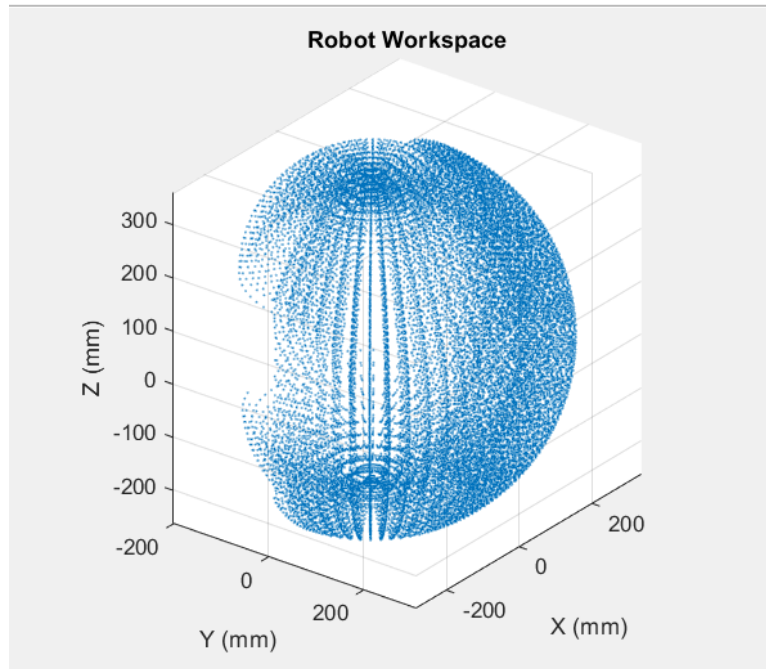


Figure 6: Computed robot workspace obtained by sampling the permitted joint-angle ranges.

6. 3D Trajectory Animation

A real-time 3D animation was developed in MATLAB to visualize the end-effector motion during simulation. The animation traces the Cartesian path of the end-effector while the robot executes the planned polynomial trajectory, allowing the motion behavior to be checked before hardware implementation.

Multiple trajectory shapes were tested to validate the trajectory-generation and IK pipeline:

- Star-shaped trajectory, used to test sharp directional changes while maintaining smooth polynomial interpolation
- Heart-shaped trajectory, used to evaluate continuous curved-path execution
- Custom waypoint paths, used to confirm general-purpose trajectory planning capability

The animation confirmed that the IK solver tracked the defined waypoints accurately and that polynomial interpolation produced continuous motion between consecutive trajectory points.

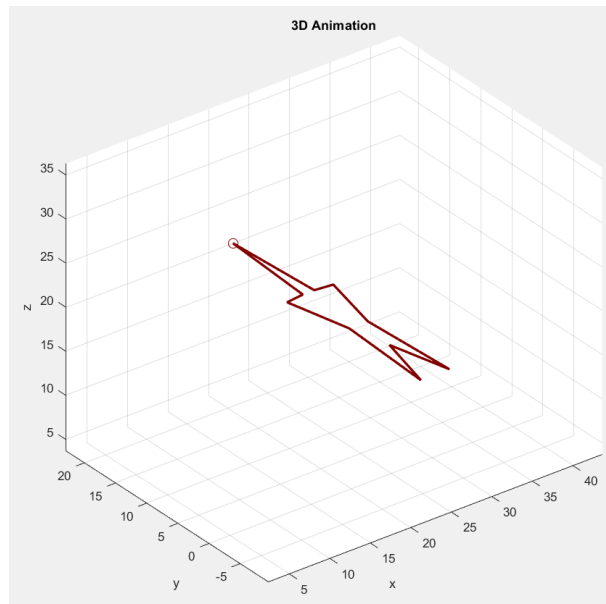


Figure 7: MATLAB 3D animation showing the generated end-effector trajectory path.

7. ROS2 Jazzy and MoveIt2 Integration

7.1 URDF Model

The robotic arm was configured as a URDF (Unified Robot Description Format) model to enable ROS2 compatibility. The URDF model defines the robot links, joint types, joint axes, inertial properties, visual geometry, and collision meshes derived from the SolidWorks design.

7.2 MoveIt2 Setup

MoveIt2 was configured using the MoveIt Setup Assistant. This process generated the SRDF (Semantic Robot Description Format), planning groups, joint limits, controller interfaces, and kinematics solver configuration. The KDL kinematics plugin was used to support inverse-kinematics calculations during motion planning.

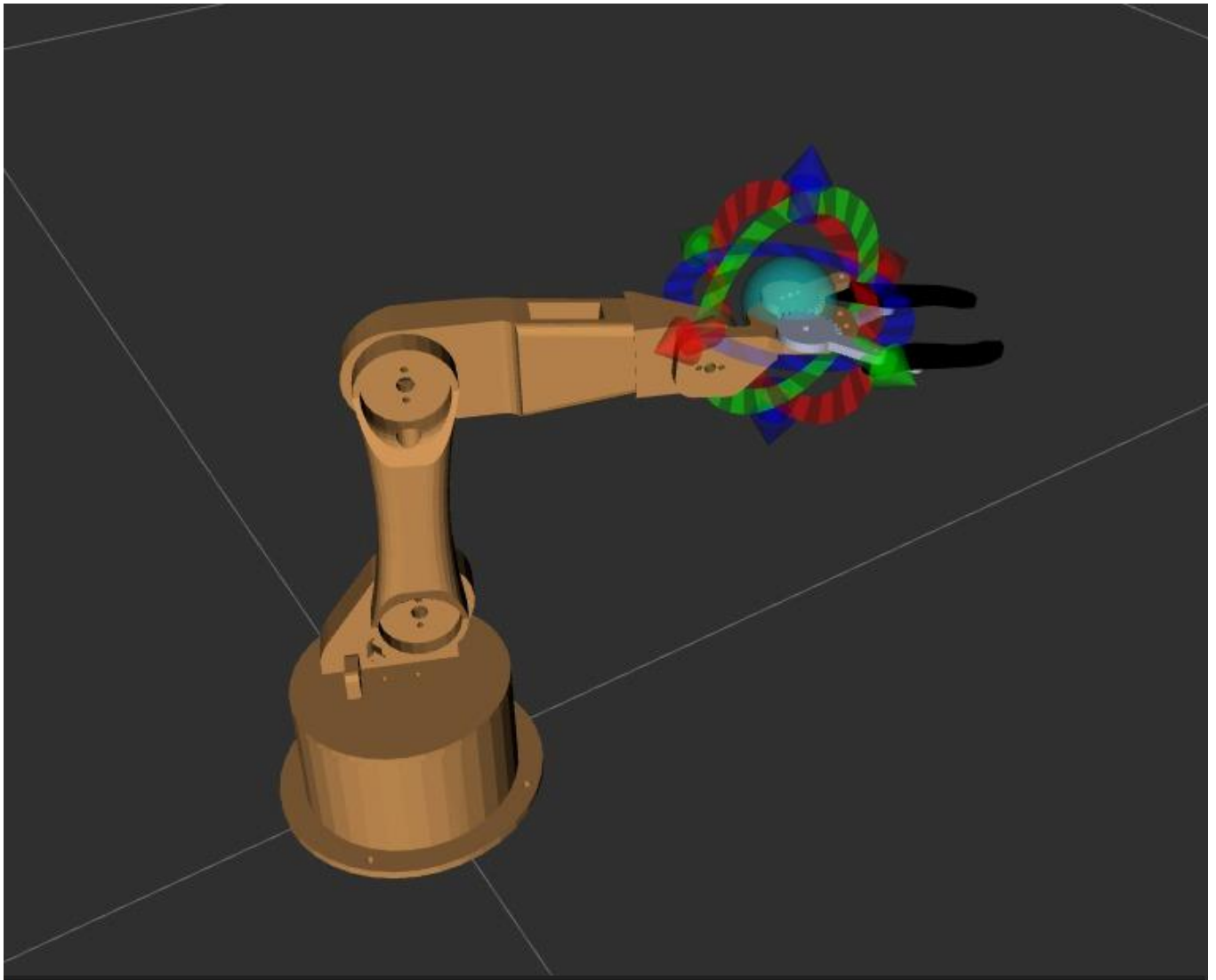
7.3 Real-Time Simulation in RViz2

The robot was visualized and interactively controlled in RViz2 using the MoveIt2 MotionPlanning plugin. The joints panel was used to define target joint states, while MoveIt2 generated collision-free trajectories to reach the selected configurations in real time.

The following joint states were observed during one of the validation tests:

- Shoulder: -38 degrees
- Elbow: -43 degrees
- Wrist: -4 degrees
- Rolling: +11 degrees
- Pitch: 0 degrees
- Gripper: -24 degrees

The MoveIt2 simulation demonstrated successful real-time motion planning with correct joint tracking and smooth trajectory execution in the virtual environment.



FilePanelsHelp

InteractMove CameraSelect

Displays

Global Options
Global Status: Ok
Grid
MotionPlanning
Status: Ok
Move Group Namespace

AddDuplicateRemoveRename

MotionPlanning

ContextPlanningJointsScene ObjectsStored ScenesStored StatesStatus

Group joints of goal state

Joint Name	Value
shoulder	-39°
elbow	-43°
wrist	-4°
rolling	11°
pitch	0°
gripper	-24°

Nullspace exploration:

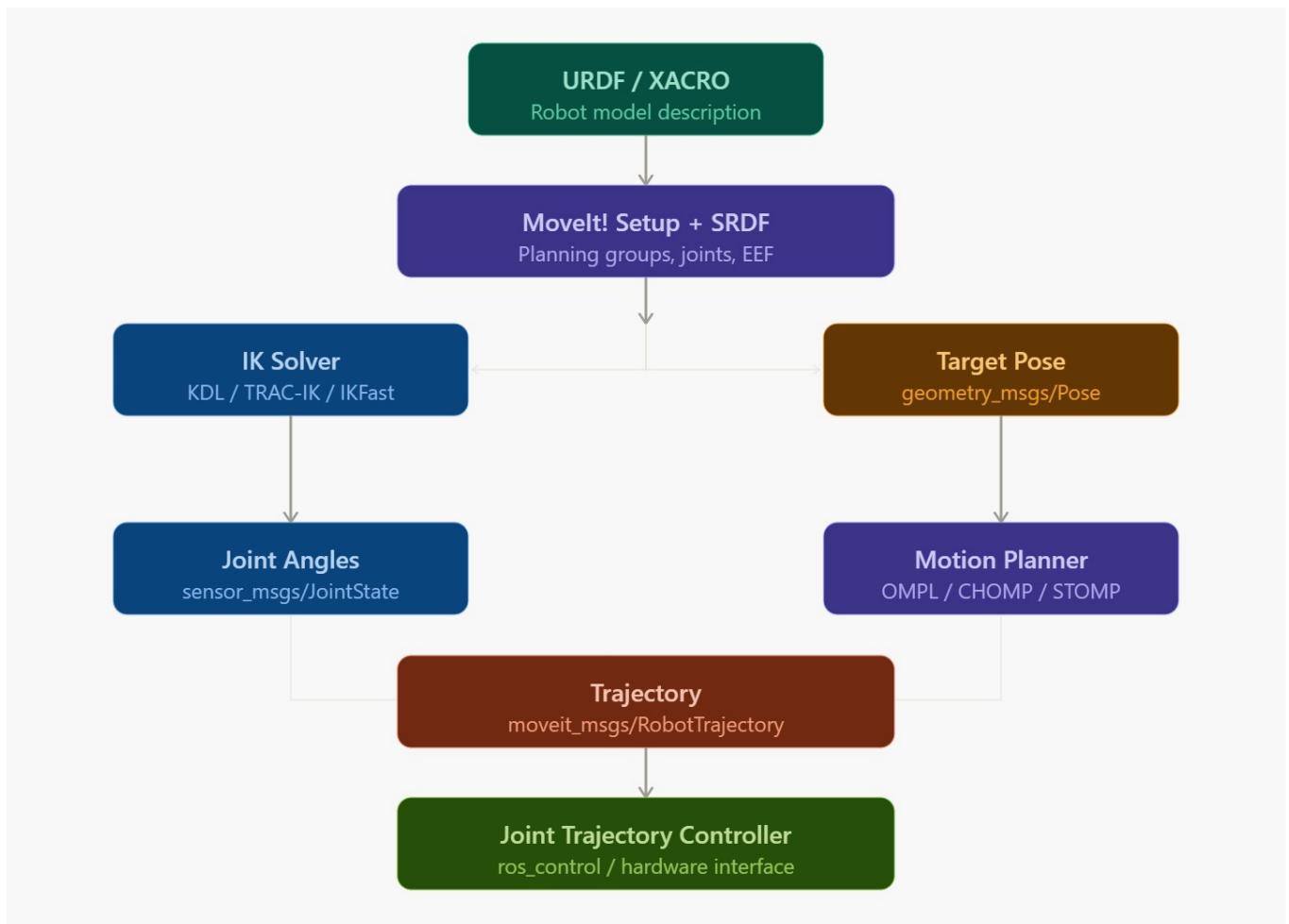
Views

Type: Orbit (rviz_default_plugins)Zero

Current View
Near Clip Distance0.01
Invert Z Axis
Target Framebase link

Orbit (rviz_default_plugins)

SaveRemoveRename



8. Custom PCB Control Board

8.1 Board Overview

A custom printed circuit board (PCB) was designed to provide a clean and reliable interface between the ESP32-WROOM microcontroller and the servo motors. The board was designed by Khaled Elwan at Alexandria University and was developed to simplify wiring, improve power distribution, and reduce connection errors during hardware testing.

The PCB design includes the following components and features:

- ESP32-WROOM-30 DevBoard used as the main microcontroller for PWM signal generation
- Six-channel servo header connectors providing signal, +5 V, and GND rails
- 220 uF electrolytic capacitor for servo power-rail filtering and voltage stabilization
- Fuse protection for the servo supply rail to reduce overcurrent risk
- Power-jack connector for external 5 V servo supply input
- LED power indicator with a 220-ohm current-limiting resistor
- Dedicated VCC and GND planes for improved power distribution and reduced wiring complexity

8.2 Servo Pin Mapping

Servo	ESP32 Pin	Function
SERVO1	D13	Shoulder joint
SERVO2	D12	Elbow joint
SERVO3	D14	Wrist joint
SERVO4	D27	Rolling joint

SERVO5	D26	Pitch joint
SERVO6	D25	Gripper

8.3 PCB Schematic

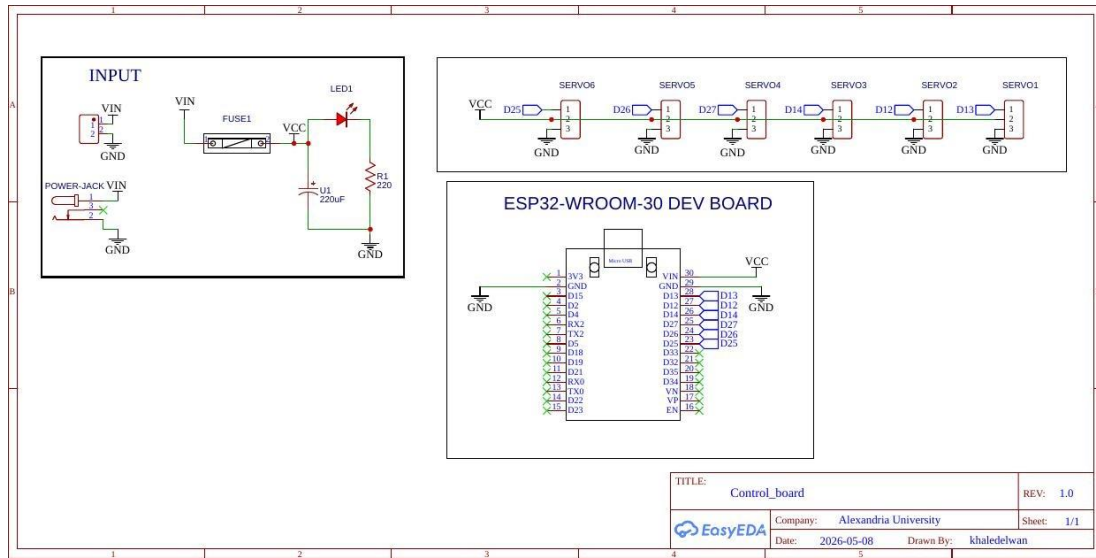


Figure 8: Control board schematic - ESP32-WROOM with six servo channels, power filtering, and fuse protection.

8.4 Assembled PCB

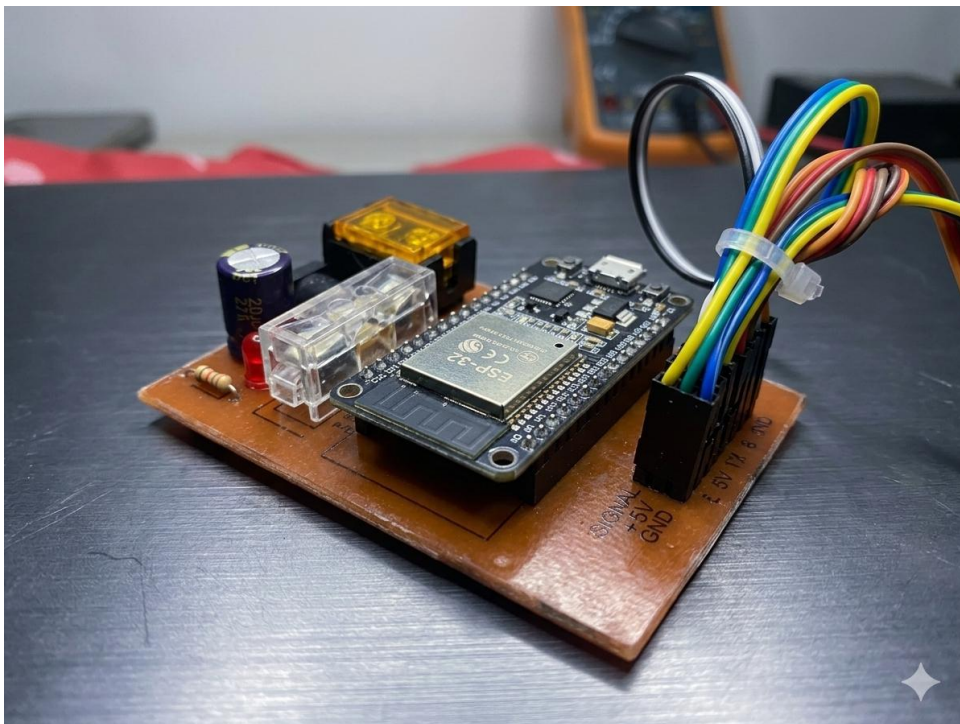


Figure 9: Assembled control board - ESP32-WROOM mounted on the custom PCB with servo connectors and power components.

9. Hardware Implementation

9.1 Control Architecture

The hardware control pipeline uses MATLAB as the high-level trajectory planner and the ESP32-WROOM as the low-level servo driver. MATLAB computes the required joint angles using the simulation and IK pipeline, then streams the command values to the ESP32 through a USB connection using the

MATLAB Support Package for Arduino Hardware. This approach eliminates the need to store the complete trajectory dataset on the microcontroller.

9.2 Control Pipeline

- MATLAB executes the Simscape simulation and IK solver to compute the joint-angle commands
- Joint angles are extracted in radians from the To Workspace blocks
- The angle values are converted from radians to degrees using rad2deg()
- The degree values are mapped from the mechanical range [-90 degrees, +90 degrees] to the normalized servo command range [0, 1]
- MATLAB streams the normalized command values to the ESP32 through the Arduino Support Package on COM9
- The ESP32 generates the servo PWM signals on pins D13, D12, D14, D27, and D26

9.3 Angle Mapping Formula

The servo writePosition() function accepts normalized command values from 0 to 1, where 0 corresponds to 0 degrees and 1 corresponds to 180 degrees. The joint angles obtained from the IK solver are therefore mapped using the following relation:

$$\text{servo_position} = (\text{joint_angle_deg} + 90) / 180$$

Joint Angle (deg)	Servo Position	Servo Angle
-90 degrees	0.0	0 degrees
0 degrees	0.5	90 degrees
+90 degrees	1.0	180 degrees

9.4 RAM Optimization

A key implementation decision was to avoid deploying the full trajectory dataset to the ESP32 memory. The trajectory contained more than 4000 time steps, corresponding to approximately 157 kB of data. When combined with the runtime stack and required libraries, this memory demand was not suitable for direct storage and execution on the ESP32.

Instead, MATLAB streams one joint-angle vector per time step at a frequency of 50 Hz, corresponding to a 20 ms update interval. This approach keeps the ESP32 memory footprint low and allows the microcontroller to operate as a real-time PWM signal generator while MATLAB performs the computationally intensive trajectory and kinematics calculations.

10. Results and Discussion

The integrated system successfully demonstrated an end-to-end robotic arm development pipeline. The mechanical design was imported into Simscape, simulated using MATLAB and Simulink, connected to trajectory planning and IK computation, visualized through workspace and 3D animation tools, and finally linked to servo control through the ESP32-based hardware platform.

The main project objectives and their completion status are summarized below:

Objective	Status
SolidWorks 3D mechanical design	Completed
Simscape Multibody physics simulation	Completed
Polynomial trajectory planning	Completed
Inverse Kinematics (IK) solver integration	Completed

3D end-effector path animation (star and heart trajectories)	Completed
Robot workspace visualization	Completed
ROS2 Jazzy URDF model	Completed
Movelt2 real-time motion planning	Completed
Custom PCB control board design	Completed
ESP32 servo hardware control	Completed

11. Conclusion

This project successfully delivered a complete robotic arm system that integrates mechanical design, physics-based simulation, trajectory planning, motion planning, custom PCB hardware, and real-time servo control. The combined workflow involving SolidWorks, MATLAB Simscape Multibody, ROS2 MoveIt2, and ESP32 demonstrates a structured and professional robotics development approach.

The polynomial trajectory planner provides smooth and configurable motion paths between Cartesian waypoints, while the IK solver calculates the corresponding joint angles required to execute the motion. The Simscape model enables physics-based validation of the robotic arm, and the MoveIt2 integration supports advanced motion planning with collision-aware trajectory generation. The custom PCB provides an organized and reliable hardware interface for controlling the servo motors.

Future enhancements may include integrating a camera module for visual feedback, implementing Gazebo-based physics simulation for pre-deployment testing, and adding a PlayStation controller interface for intuitive teleoperation and manual control.

References

[1] MATLAB Robotics System Toolbox Documentation, MathWorks, R2024a.
[2] Simscape Multibody User Guide, MathWorks, R2024a.
[3] MoveIt2 Documentation, Open Robotics, 2024. <https://moveit.ros.org>
[4] ROS2 Jazzy Documentation, Open Robotics, 2024. <https://docs.ros.org/en/jazzy>
[5] J. J. Craig, Introduction to Robotics: Mechanics and Control, 3rd ed., Pearson, 2005.
[6] ESP32 Technical Reference Manual, Espressif Systems, 2024.
[7] SolidWorks Documentation, Dassault Systemes, 2024.